

Data Architecture

Project: Baby Milk Tracker / MilkWise

Author: Kit

Date: 2026-06-15

Status: Reference document

1. Overview

The app uses **no database**. All persistence is three plain JSON files on the Pi filesystem, written and read by the Next.js API routes via Node.js `fs`. All numbers displayed in the UI are computed on the fly from these files; nothing displayed is stored directly.

```
/data/
  feeds.json      ← array of Feed objects
  settings.json   ← single Settings object
  weights.json    ← array of WeightEntry objects
```

The path is controlled by the `DATA_DIR` environment variable (default: `../..data` relative to the app root, placing it outside the Next.js project so Turbopack does not watch it).

2. Stored Data

2.1 feeds.json

An array of `Feed` objects. Each represents one feeding event.

Field	Type	Description
<code>id</code>	string	Unique identifier (UUID v4 or timestamp+random)
<code>timestamp</code>	number	Unix milliseconds — when the feed started
<code>volume</code>	number	Water ml measured into the bottle
<code>targetMlPerDay</code>	number?	~~Daily milk target active at log time.~~ Deprecated — do not write. This field was a workaround from before <code>weights.json</code> existed. The target at any

Field	Type	Description
		past time is derivable from weight history via <code>dailyTargetAtTime()</code> . Existing entries may still carry the field; it is ignored in all new calculations.

Important: `volume` is always stored in **water ml**, not formula ml. The conversion to formula ml happens at read time using the lookup table in `calculations.ts`. Never store formula ml directly.

Immutability principle: The only immutable facts about a feed are `id`, `timestamp`, and `volume`. Everything else (what the target was, what the weight was, what the hourly rate was) is derivable and must not be stored.

Current live state (idea02): **143 feed entries**, spanning 2026-05-31 to 2026-06-15.

2.2 settings.json

A single `Settings` object. Always read as a merge over `DEFAULT_SETTINGS` so missing fields get defaults.

Field	Type	Default	Description
<code>weightKg</code>	number	6.27	Current baby weight in kg (kept in sync with latest weight entry)
<code>mlPerKgPerDay</code>	number	150	WHO recommended formula intake rate
<code>standardBottleVolume</code>	number	90	Water ml — standard bottle size for DailyTargetCard pictograms
<code>displayBottleVolumeWater</code>	number	90	Water ml — bottle size used for "bottles" count in StatusCard
<code>yellowThresholdPct</code>	number	5	±% deviation before status turns yellow
<code>redThresholdPct</code>	number	10	±% deviation before status turns red
<code>timeFormat</code>		'24h'	'24h'

Field	Type	Default	Description
<code>maxCorrectionPct</code>	number	25	Max shift of adjusted next feed as % of ideal interval
<code>useTargetAwarePredictor</code>	boolean	true	true = Predictor 3 (T*); false = Predictor 2 (Formula S)
<code>nextBottleWaterMl</code>	number	90	Water ml — assumed next bottle size for Predictor 3

`settings.weightKg` is automatically kept in sync by the weights POST route: whenever a new weight entry is saved, `settings.weightKg` is updated to the most recent weight. **This makes `settings.weightKg` a derived cache, not a primary fact.** It should be treated as a legacy field and will be superseded by the weight history once onboarding is implemented (see §2.3). Code must not use `settings.weightKg` as the authoritative weight for any calculation; it should use `effectiveWeightAtTime(t, weights)` instead.

2.3 weights.json

An array of `WeightEntry` objects, one per weigh-in.

Field	Type	Description
<code>id</code>	string	Unique identifier
<code>timestamp</code>	number	Unix milliseconds — when this weight was measured
<code>weightKg</code>	number	Baby's weight in kg at this time

Current live state (idea02): **11 weight entries**, spanning 2026-04-20 to 2026-06-15. Weights range from 4.13 kg (birth area) to 6.93 kg (latest). Data persists via Docker bind mount at `/instances/milkwise-idea02-001/data/weights.json` on idea02 — survives container restarts.

Note: The dev workspace copy on wizardly-hugle has no `weights.json` (no entries logged there). The live instance on idea02 is the production data store.

2.4 Onboarding invariant

From the next implementation version onward, `weights.json` is **guaranteed non-empty** at runtime. Onboarding (triggered when `weights.length === 0`) requires the parent to enter a first weight before the dashboard is accessible. This removes the need for any fallback weight value and makes `settings.weightKg` redundant as a calculation input.

The onboarding trigger condition: `weights.length === 0` on first dashboard render → redirect to `/onboarding`.

2.5 Code-level constants (not in any file)

Several values are baked into the source code rather than stored:

Constant	Location	Value	Description
<code>FORMULA_TABLE</code>	<code>calculations.ts</code>	7 pairs	Water→formula ml lookup table per manufacturer's ratio
<code>MIN_DEFICIT_ML</code>	<code>calculations.ts</code>	15 ml	Minimum deficit before bottle recommendation activates
<code>MAX_WATER / MAX_FORMULA</code>	<code>calculations.ts</code>	150 / 170 ml	Largest practical bottle size
Credit decay formula	<code>calculations.ts</code>	<code>max(0, milk - hourlyRate * (age - 24))</code>	How a bottle's contribution decays after 24h

3. Core Derived Quantities

These are the fundamental computed values from which all displayed numbers ultimately derive. They are recomputed on every page load.

3.1 Formula conversion: `waterToMilk(waterMI) → formulaMI`

The single most important conversion in the app. Logged volumes are in **water ml** (what you pour). All energy calculations use **formula ml** (what the baby actually gets, including the powder).

The conversion is **non-linear** — it uses a lookup table interpolated linearly between entries:

Water ml	Formula ml
30	35
60	70
90	100
120	135
150	170
180	200
210	240

Outside the table range, the nearest segment slope is extrapolated. A 90 ml water bottle → **100 ml formula** (the most common case).

3.2 Derived settings (from Settings + weight history)

Current (pre-compensation) derivation:

```
dailyTargetMl      = settings.weightKg × mlPerKgPerDay  (formula
ml/day)
hourlyRate         = dailyTargetMl / 24                (formula
ml/hour)
milkPerBottle      = waterToMilk(standardBottleVolume) (formula
ml per standard bottle)
idealIntervalHours = milkPerBottle / hourlyRate        (hours
between feeds)
```

Post-compensation (correct) derivation:

```
effectiveWeight    = effectiveWeightAtTime(t, weights)  (kg – from
weight history, NOT settings)
dailyTargetMl(t)  = effectiveWeight(t) × mlPerKgPerDay (time-
dependent – changes as baby grows)
hourlyRate(t)     = dailyTargetMl(t) / 24
milkPerBottle      = waterToMilk(standardBottleVolume)
(unchanged)
idealIntervalHours(t) = milkPerBottle / hourlyRate(t)
```

Critical: `dailyTargetMl` is not a fixed session constant — it is time-dependent. A calculation evaluating historical data must use the weight at

the historical time, not a current snapshot. Code that calls `deriveSettings(settings)` once and reuses `derived.dailyTargetMl` throughout the session will produce incorrect values for historical calculations after the baby grows.

At current live data (6.93 kg, 150 ml/kg/day, 90 ml standard bottle): -
`dailyTargetMl` = 1039.5 ml/day - `hourlyRate` = 43.31 ml/hour -
`milkPerBottle` = 100 ml - `idealIntervalHours` = 2.31 hours

3.3 Reference time for frozen calculations

A critical design principle: status numbers are **frozen at** `lastFeed.timestamp`, not at "now". This means if a parent checks the app 4 hours after the last feed, the status still shows what state the baby was in at that feed — not a decayed version.

```
lastFeed      = most recent Feed (by timestamp)
smoothedAt    = lastFeed.timestamp  (if any feeds exist;
otherwise now)
```

3.4 Strict 24h total

All formula ml from feeds whose timestamp falls within the 24-hour window ending at `smoothedAt`:

```
cutoff = smoothedAt - 24 × 60 × 60 × 1000
strict24h =  $\sum$  waterToMilk(feed.volume)  for all feeds with
timestamp  $\geq$  cutoff
```

3.5 Bottle credit (the smoothed model)

Each feed contributes "credit" that decays after 24 hours:

```
ageHours = (refTime - feed.timestamp) / 3_600_000
credit = waterToMilk(feed.volume)                if ageHours  $\leq$ 
24
credit = max(0, milk - hourlyRate × (ageHours - 24)) if ageHours >
24
```

The intuition: a bottle fully counts for 24 hours, then its contribution decays at the hourly rate until it reaches zero.

3.6 Smoothed total (frozen at lastFeed)

```
smoothedMl =  $\sum$  credit(feed, hourlyRate, smoothedAt)    over ALL  
feeds
```

This is the primary energy balance metric. It is evaluated at `smoothedAt = lastFeed.timestamp`.

3.7 Live smoothed total (ticking with clock)

Same formula as 3.6 but evaluated at `now` (updated every 60 seconds):

```
liveSmoothedMl =  $\sum$  credit(feed, hourlyRate, now)
```

3.8 Percentages

```
strict24hPct    = (strict24h    / dailyTargetMl) × 100  
smoothedPct     = (smoothedMl   / dailyTargetMl) × 100  
liveSmoothedPct = (liveSmoothedMl / dailyTargetMl) × 100
```

3.9 Status colour

```
diff = |pct - 100|  
colour = green    if diff ≤ yellowThresholdPct  
colour = yellow   if diff ≤ redThresholdPct  
colour = red      otherwise
```

3.10 Standard next feed time

```
nextBottleMilkMl  = waterToMilk(standardBottleWaterMl)  
standardIntervalMs = (nextBottleMilkMl / hourlyRate) × 3_600_000  
standardNext      = lastFeed.timestamp + standardIntervalMs
```

This is the naive "when will this bottle be used up" time. It uses the selected next-bottle size, not the last feed size.

3.11 Adjusted next feed time (Predictor 3 — T*)

Binary search for the time T* at which:

```
smoothedAtTime(T*) = dailyTargetMl - nextBottleMilkMl
```

The idea: find the moment at which giving a standard bottle would bring the smoothed total exactly to `dailyTargetMl`. The search is clamped to `[lastFeed.timestamp, standardNext + maxCorrectionMs]`. If the baby is already underfed, `T = lastFeed.timestamp` (feed now). If still overfed at the cap, `T = cap` and `capped = true`.

3.12 Best bottle size now

```
deficitMl = dailyTargetMl - smoothedAtTime(now)
```

If `deficitMl ≤ 0` or `< 15 ml`: return "overfed", recommend waiting.
Otherwise: snap `deficitMl` to the nearest entry in `FORMULA_TABLE` (max 170 ml formula = 150 ml water), return that entry as the recommended bottle.

3.13 Trend points (analytics)

For each feed within the selected window:

```
target(feed) = dailyTargetAtTime(feed.timestamp, weights,  
mlPerKgPerDay, fallback)  
hr(feed)     = target(feed) / 24  
smoothed(feed) = Σ credit(f, hr(feed), feed.timestamp) over ALL  
feeds  
surplus(feed) = smoothed(feed) - target(feed)
```

Each feed becomes one point on the trend graph, plotted as surplus (positive = overfed, negative = underfed).

3.14 Analytics statistics

```
avgIntervalHours = mean of all inter-feed gaps (sorted feeds,  
consecutive pairs)  
consistencyScore = σ (standard deviation) of inter-feed gaps  
periodTotal(days) = Σ feed.volume (water ml) for feeds within
```



```
last N days          (note: this is raw water ml – a deliberate
raw count)
avgSurplusMl        = mean surplus over trend points in selected
window
```

3.15 Weight analytics (WeightChart)

```
gainKg      = latest.weightKg - oldest.weightKg
daySpan     = (latest.timestamp - oldest.timestamp) / 86_400_000
gainPerWeek = (gainKg / daySpan) × 7    (if daySpan > 0)
```

4. Screen-by-Screen UI Element Reference

4.1 Dashboard (page.tsx)

Header

Element	Data
" Baby Milk Tracker"	Static label
Subtitle: 6.47 kg · Target: 970 ml/day	<code>settings.weightKg · round(dailyTargetMl)</code>

DailyTargetCard (swipeable, 4 views)

View 0 — Numeric (default):

Element	Derived from
Large ml number	<code>round(dailyTargetMl)</code>
Bottle count	<code>dailyTargetMl / waterToMilk(displayBottleVolumeWater)</code>
"× 90ml bottles" label	<code>displayBottleVolumeWater</code> from settings
"every Xh Ym"	<code>idealIntervalHours</code> (using <code>displayBottleVolumeWater</code>)
Formula line below	<code>weightKg × mlPerKgPerDay</code>

Element	Derived from
Bottle pictograms	Each = one standard bottle; opacity of partial = fractional remainder
Pictogram label (ml)	Full bottles: <code>standardBottleVolume</code> ; partial: <code>round(partial × milkPerBottle)</code>

View 1 — Bottle Parade:

Element	Derived from
icons	<code>floor(dailyTargetMl / milkPerBottle)</code> full + partial
"X.X bottles of Y ml"	<code>totalBottles.toFixed(1), standardBottleVolume</code>
"X ml target"	<code>round(dailyTargetMl)</code>

View 2 — Feed Clock:

Element	Derived from
Time chips	<code>feedsPerDay = 24 / idealIntervalHours</code> times starting at 00:00, spaced by <code>idealIntervalHours</code>
"Every X.Xh"	<code>idealIntervalHours</code>
"~N feeds/day"	<code>round(24 / idealIntervalHours)</code>

View 3 — Fun Facts:

Element	Derived from
"X.X cans of soda"	<code>dailyTargetMl / 330</code>
"N cups of tea"	<code>round(dailyTargetMl / 150)</code>
"X.XX litres"	<code>dailyTargetMl / 1000</code>
"X.X jam jars"	<code>dailyTargetMl / 500</code>

StatusCard (swipeable, 12 views)

All views share the same underlying data: `smoothedMl`, `smoothedPct`, `liveSmoothedMl`, `liveSmoothedPct`, `strict24h`, `strictPct`, `dailyTargetMl`, and `feeds24h` (feeds from the last 24h window before now).

View 0 — Status at last feed (PanelWithGauge — default):

Element	Derived from
Large ml number	<code>round(smoothedMl)</code>
Percentage	<code>round(smoothedPct)</code>
Status text ("on track", "slightly over", etc.)	<code>statusText(smoothedPct, yellow, red)</code>
Border/background colour	<code>bgBorder(smoothedPct, yellow, red)</code>
Gauge bar fill height	<code>(smoothedPct - 60) / 80 × 100</code> , clamped 0-100%
Gauge delta label ("+Xml")	<code> round(smoothedPct/100 × dailyTargetMl - dailyTargetMl) </code>
pictograms	One per feed in <code>feeds24h</code> ; font-size scaled by <code>waterToMilk(volume) / milkPerBottle</code>
Pictogram label (ml)	<code>feed.volume</code> (water ml, as logged)

View 1 — Status now (PanelWithGaugeLive):

Same layout as View 0 but all values use `liveSmoothedMl` and `liveSmoothedPct` instead. The gauge animates with CSS transition on each 60s clock tick.

View 2 — Dual gauge (PanelDualGauge):

Two side-by-side gauges: left = `smoothedPct` ("at feed"), right = `liveSmoothedPct` ("now"). Pictograms from `feeds24h`.

View 3 — Intake trend 3 days (FeedTrendView):

Canvas graph. Each dot = one feed within the 3-day window; y-axis = `surplus` (from §3.13). Curve = Catmull-Rom spline through dots. Colour zones from `yellowThresholdPct` / `redThresholdPct`. Percentage badge = `round(smoothedPct)`.

View 4 — STATUS LAST 24H (Panel, smoothed):

`smoothedMl`, `smoothedPct`, bottle count = `smoothedMl / milkPerBottle`, pictograms from `feeds24h`.

View 5 — Strict 24h (Panel):

`strict24h`, `strictPct`, bottle count = `strict24h / milkPerBottle`, pictograms from `feeds24h`.

View 6 — Progress bars (ProgressView):

Two horizontal bars: Smoothed (`smoothedMl`, `smoothedPct`) and Strict (`strict24h`, `strictPct`). Bar fill = `min(pct, 150) / 150 × 100%`. White tick at 66.7% of bar width = 100% target. Footer: `round(dailyTargetMl)`.

View 7 — At a glance (SpotlightView):

Left: `round(smoothedPct)%`, `round(smoothedMl)` ml. Right: `round(strictPct)%`, `round(strict24h)` ml.

View 8 — Balance bar (BiDirectionalView):

Centre-anchored bar. Fill extends left (underfed, blue) or right (overfed, orange). `diff = smoothedPct - 100`. `surplusMl = |round(smoothedMl - dailyTargetMl)|`. Bar fill = `min(|diff| / 40 × 50, 50)%`.

View 9 — Intake gauge (ThermometerView):

Vertical thermometer. Fill = `(smoothedPct - 60) / 80 × 100%`, clamped. Delta: `round(|diff|)%` and `surplusMl` ml.

View 10 — Emoji status (EmojiBalanceView):

Emoji and text chosen by `diff = smoothedPct - 100` and thresholds. `surplusMl = |round(smoothedMl - dailyTargetMl)|`.

View 11 — History link (HistoryLinkView):

Shows `round(smoothedMl)` ml and `round(smoothedPct)%`. Link to `/history/smoothed`.

Three-clock row (Last feed / Next / Adjusted)

Element	Derived from
Last feed timestamp	<code>lastFeed.timestamp</code>
Last feed volume	<code>lastFeed.volume</code> (water ml)
Next timestamp	<code>standardNext = lastFeed.timestamp + (waterToMilk(standardBottleWaterMl) / hourlyRate) × 3_600_000</code>
Next relative label ("in 2h 15m")	<code>standardNext - now</code> , formatted
Next bottle pill (🍼 90ml)	<code>standardBottleWaterMl</code> state variable (user-selectable; defaults to <code>settings.nextBottleWaterMl</code>)
Adjusted timestamp	T* from Predictor 3 (\$3.11)
Adjusted relative label	<code>nextFeed - now</code> , formatted
Adjusted "+Nm later / -Nm earlier"	<code>round((nextFeed - standardNext) / 60_000)</code> minutes

Element	Derived from
Adjusted bottle pill	<code>nextBottleWaterMl</code> state variable
Adjusted "Feed now → X ml water"	<code>bestBottleSizeNow(feeds, hourlyRate, dailyTargetMl, now)</code> (\$3.12)
Adjusted [⚠] Over target — wait until HH:MM"	Shown when <code>bestBottleSizeNow.status === "overfed"</code> and <code>nextFeed</code> exists

Summary row

Element	Derived from
Total feeds	<code>feeds.length</code>
Last 24h feeds	<code>feeds.filter(f => f.timestamp >= now - 86_400_000).length</code>
ml/hour	<code>round(hourlyRate × 10) / 10</code>

4.2 Log Feed (/log)

Element	Derived from
Bottle size buttons (30/60/90/120/150 ml)	Fixed constants <code>QUICK_VOLUMES</code>
"90 ml water → 100 ml milk" conversion hint	<code>waterToMilk(bottleSize)</code>
Pre-filled volume field	<code>round(waterToMilk(bottleSize))</code> formula ml
Recommendation message ([?] <code>recommend=X</code>)	<code>bestBottleSizeNow</code> result passed via URL param from dashboard
Date field	<code>new Date()</code> formatted as YYYY-MM-DD; refreshes every 30s if not edited
Time field	<code>new Date()</code> formatted as HH:MM; refreshes every 30s if not edited
Recent feeds list	Last 3 feeds sorted by timestamp descending; shows <code>feed.timestamp</code> (formatted) and <code>feed.volume</code> (water ml)

On save: `volume` (formula ml shown in input) is converted back to water ml via `milkToWater()` before writing. `targetMlPerDay` is stamped as `round(settings.weightKg × settings.mlPerKgPerDay)`.

4.3 History (/history) — Feeds tab

Element	Derived from
Feed list (sorted newest first)	All <code>feeds</code> from <code>feedsWithCredit(feeds, hourlyRate)</code>
Date/time	<code>feed.timestamp</code> formatted
Volume (blue)	<code>feed.volume</code> (water ml as logged)
Age	$(\text{now} - \text{feed.timestamp}) / 3_600_000$ hours
Credit	<code>bottleCredit(ageHours, waterToMilk(feed.volume), hourlyRate)</code> — formula ml remaining
Search filter	Matches <code>feed.timestamp</code> against date string input

4.4 History (/history) — Weight tab

Element	Derived from
Weight entries (sorted newest first)	<code>weights</code> array
Weight value	<code>weight.weightKg</code>
Date/time	<code>weight.timestamp</code> formatted

4.5 Analytics (/analytics) — Intake tab

Trend chart (TrendCanvas)

Element	Derived from
Each dot	One feed within window; y-position = <code>surplus</code> (\$3.13)
Dot colour	Green if $ \text{surplus} \leq \text{dailyTarget} \times \text{yellowPct} / 100$; yellow if within red threshold; red/blue if beyond
Curve	Catmull-Rom spline through all dots
Green zone band	$\pm \text{dailyTargetMl} \times \text{yellowThresholdPct} / 100$ around zero $\pm \text{dailyTargetMl} \times \text{redThresholdPct} / 100$

Element	Derived from
Yellow zone band	
Zero line	<code>surplus = 0</code> = exactly on daily target
Day separators	Midnight boundaries within the window
Period buttons (3d/7d/30d)	Sets <code>windowMs = days × 86_400_000</code>

Statistics grid

Element	Derived from
Avg interval	<code>avgIntervalHours</code> (§3.14)
Ideal interval (sub-label)	<code>idealIntervalHours</code>
Consistency (σ)	<code>consistencyScore</code> (§3.14) — std dev of inter-feed gaps
Avg Nd surplus	<code>mean(trendPoint.surplus)</code> for all points in window
Total feeds	<code>feeds.length</code>

Period totals

Element	Derived from
Last 3/7/14 days (ml)	<code>periodTotal(feeds, N) = Σ feed.volume</code> (raw water ml) within window

Note: Period totals are in raw water ml, not formula ml. This is intentional — it shows raw intake, not converted.

Consistency explainer modal

Shows `avgIntervalHours`, `consistencyScore`, `idealIntervalHours` from §3.14.

4.6 Analytics (/analytics) — Weight tab

Statistics grid (WeightChart header)

Element	Derived from
Current weight	<code>sorted.last.weightKg</code>

Element	Derived from
Total gain	<code>latest.weightKg - oldest.weightKg</code>
Rate (g/week)	<code>(gainKg / daySpan) × 7 × 1000</code>

Weight chart (canvas)

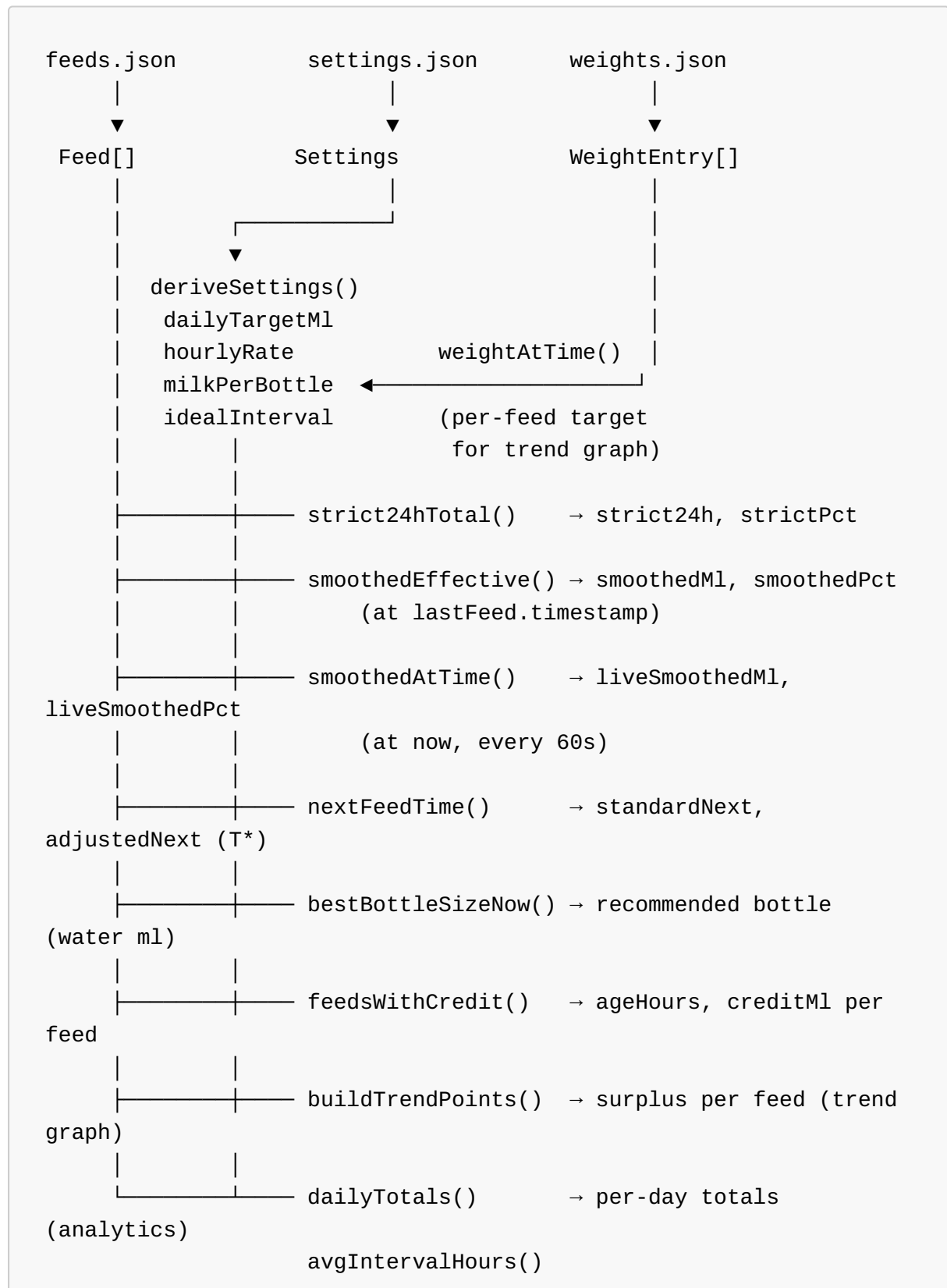
Element	Derived from
Each dot	<code>weight.weightKg</code> at <code>weight.timestamp</code>
Curve	Catmull-Rom spline through sorted weight entries
Dot label	<code>weight.weightKg</code> kg
Y axis	Auto-scaled: <code>min(weightKg) - 0.1</code> to <code>max(weightKg) + 0.1</code>
X axis	<code>sorted.first.timestamp</code> to <code>sorted.last.timestamp + 1 day</code>
Grid lines	Daily or weekly ticks depending on span

4.7 Settings (/settings)

Element	Derived from / writes to
Baby weight field	Reads/writes <code>settings.weightKg</code>
ml per kg per day	Reads/writes <code>settings.mlPerKgPerDay</code>
On-track zone (±%)	Reads/writes <code>settings.yellowThresholdPct</code>
Seriously off threshold	Reads/writes <code>settings.redThresholdPct</code>
Display bottle size buttons	Reads/writes <code>settings.displayBottleVolumeWater</code>
Max correction (%)	Reads/writes <code>settings.maxCorrectionPct</code>
Predictor toggle	Reads/writes <code>settings.useTargetAwarePredictor</code>
Time format	Reads/writes <code>settings.timeFormat</code>
Auto-calculated: Daily target	<code>weightKg × mlPerKgPerDay</code>
Auto-calculated: Hourly rate	<code>dailyTargetMl / 24</code>
Auto-calculated: Ideal interval	<code>idealIntervalHours</code>

Note: the auto-calculated panel uses `deriveSettings(settings)` with the current live form values, not the saved values. It updates immediately as the user types.

5. Data Flow Summary



```
consistencyScore()
periodTotal()
```

6. API Routes

All routes are Next.js Route Handlers (`app/api/*/route.ts`).

Method	Route	Reads	Writes
GET	<code>/api/feeds</code>	<code>feeds.json</code>	—
POST	<code>/api/feeds</code>	<code>feeds.json</code>	<code>feeds.json</code>
DELETE	<code>/api/feeds/[id]</code>	<code>feeds.json</code>	<code>feeds.json</code>
PATCH	<code>/api/feeds/[id]</code>	<code>feeds.json</code>	<code>feeds.json</code>
GET	<code>/api/settings</code>	<code>settings.json</code>	—
POST	<code>/api/settings</code>	—	<code>settings.json</code>
GET	<code>/api/weights</code>	<code>weights.json</code>	—
POST	<code>/api/weights</code>	<code>weights.json</code> , <code>settings.json</code>	<code>weights.json</code> , <code>settings.json</code>
DELETE	<code>/api/weights/[id]</code>	<code>weights.json</code>	<code>weights.json</code>
PATCH	<code>/api/weights/[id]</code>	<code>weights.json</code>	<code>weights.json</code>

The POST `/api/weights` route has a side effect: it also writes `settings.weightKg` to the most recent weight value.

7. Client-side State and Caching

The app uses React component state only — no Redux, no Zustand, no `localStorage` (`localStorage` was migrated away from). On every page load:

1. `load()` fetches `GET /api/feeds`, `GET /api/settings`, `GET /api/weights` in parallel.
2. All calculations are run synchronously in the render path.
3. A 60-second `setInterval` updates `now` (the clock) without re-fetching feeds.
4. A `window.focus` listener re-runs `load()` when the tab regains focus (handles navigation back from other pages).

There is no caching layer. Every focus event causes a full re-fetch from disk.

Onboarding gate (post-implementation): If `weights.length === 0` after `load()`, the dashboard redirects to `/onboarding` before rendering any calculated data. This guarantees all calculations have at least one weight entry to work from.